

Unit 1: Finite Automata & Regular Expressions

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Complete unit syllabus topics.

Table of Contents

Central Concepts of Automata Theory	p. 2
Deterministic Finite Automata (DFA)	p. 2
DFA Design & Worked Examples	p. 3
Nondeterministic Finite Automata (NFA)	p. 7
ε-NFA & ε-Closure	p. 8
Equivalence of DFA and NFA: Subset Construction Algorithm	p. 8
Conversion of ε-NFA to DFA	p. 12
Real-World Applications of Finite Automata	p. 12
Unit 1 Quick Revision Sheet	p. 13
Important Exam Questions	p. 14
Solved Questions from Past CIE-1 & SEE Examination Papers	p. 16
Practice Problems with Step-by-Step Solutions	p. 20

Central Concepts of Automata Theory

§ 1

Automata theory mathematically models abstract computing machines that process discrete input sequences according to rigorous state transitions.

CORE DEFINITIONS: ALPHABET, STRING, AND LANGUAGE

1. **Alphabet (Σ):** A finite, non-empty set of symbols.

Examples: $\Sigma = \{0, 1\}$ (binary), $\Sigma = \{a, b, c\}$, $\Sigma = \text{ASCII}$

2. **String (Word, w):** A finite sequence of symbols chosen from an alphabet Σ .

- **Length of a string ($|w|$):** The number of symbol positions in w (e.g., if $w = 01011$, $|w| = 5$).
- **Empty String (ϵ or Λ):** The unique string of length zero: $|\epsilon| = 0$.
- **Powers of an Alphabet (Σ^k):** The set of all strings of length k . $\Sigma^0 = \{\epsilon\}$.
- **Kleene Star (Σ^*):** The set of all strings of all possible lengths over Σ , including ϵ :

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$$

- **Positive Closure (Σ^+):** The set of all non-empty strings: $\Sigma^+ = \Sigma^* \setminus \{\epsilon\}$.

3. **Language (L):** Any subset of strings over an alphabet: $L \subseteq \Sigma^*$. A language may be finite (e.g., $L = \{0, 01, 11\}$) or infinite (e.g., $L = \{a^n b^n \mid n \geq 1\}$).

Quick Recall

Σ is an alphabet; w is a string of length $|w|$; ϵ has length 0; Σ^* contains all finite strings including ϵ ; a language L is any subset $L \subseteq \Sigma^*$.

Deterministic Finite Automata (DFA)

§ 2

A **Deterministic Finite Automaton (DFA)** is a state machine in which for each state and for every input symbol, there is *exactly one unique transition* to a next state.

FORMAL 5-TUPLE DEFINITION OF A DFA

A DFA is mathematically formalized as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

1. Q : A finite, non-empty set of states.
2. Σ : A finite alphabet of input symbols.
3. δ : The state transition function, mapping a state and an input symbol to exactly one state:

$$\delta : Q \times \Sigma \rightarrow Q$$

4. q_0 : The unique initial (start) state, where $q_0 \in Q$.
5. F : The set of final (accepting) states, where $F \subseteq Q$.

Extended Transition Function ($\hat{\delta}$)

To process an entire string $w \in \Sigma^*$ rather than an individual symbol, we define the **extended transition function** $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ recursively:

$$\text{Base Step: } \hat{\delta}(q, \epsilon) = q$$

$$\text{Inductive Step: } \hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a), \quad \text{where } x \in \Sigma^* \text{ and } a \in \Sigma$$

Language Accepted by a DFA:

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$$

Quick Recall

DFA 5-tuple: $(Q, \Sigma, \delta, q_0, F)$. Determinism requires that $\delta(q, a)$ is defined for every symbol $a \in \Sigma$, with exactly one deterministic destination state.

DFA Design & Worked Examples

§ 3

Design Problem 1 · Core Pattern**DFA Construction**

Design a DFA M that accepts the language of all strings over $\Sigma = \{0, 1\}$ that end with the substring "01":

$$L = \{w \in \{0, 1\}^* \mid w \text{ ends with } 01\}$$

Provide the formal 5-tuple, state transition table, and state diagram.

Step 1: State Formulation and Intuition

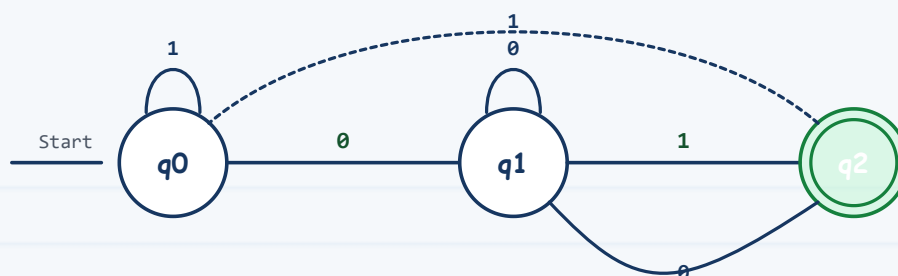
- q_0 : Start state. Represents the condition where the most recent symbol was not '0' (e.g., initial state or just read a '1').
- q_1 : The most recent symbol read was '0'. Halfway to the target '01'.
- q_2 : The most recent two symbols read were '01'. Accepting state.

Step 2: Transition Logic

- From q_0 : on '0' $\rightarrow q_1$; on '1' $\rightarrow q_0$.
- From q_1 : on '0' $\rightarrow q_1$ (still ends in '0'); on '1' $\rightarrow q_2$ (now ends in '01'!).
- From q_2 : on '0' $\rightarrow q_1$ (string now ends in '0'); on '1' $\rightarrow q_0$ (ends in '1', resetting progress).

Step 3: State Transition Table

State (Q)	Input Symbol: 0	Input Symbol: 1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

Step 4: State Transition Diagram

Formal 5-Tuple $M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$

Design Problem 2 · Exam Level**Modular Arithmetic Automata**

Construct a DFA that accepts all binary strings representing integers **divisible by 3** (reading MSB first):

$$L = \{w \in \{0, 1\}^* \mid \text{value}(w) \equiv 0 \pmod{3}\}$$

Step 1: Modular State Formulations

When appending a new bit $b \in \{0, 1\}$ to the right of an existing binary number with value V , the new value is $V_{\text{new}} = 2V + b$.

$$\text{New Remainder} = (2 \times \text{Current Remainder} + b) \pmod{3}$$

- State q_0 : Remainder 0 (mod 3) (Initial & Accepting state).
- State q_1 : Remainder 1 (mod 3).
- State q_2 : Remainder 2 (mod 3).

Step 2: Transition Computations

- From q_0 (rem 0): on 0 $\rightarrow (2(0) + 0) \% 3 = 0 \implies q_0$; on 1 $\rightarrow (2(0) + 1) \% 3 = 1 \implies q_1$.
- From q_1 (rem 1): on 0 $\rightarrow (2(1) + 0) \% 3 = 2 \implies q_2$; on 1 $\rightarrow (2(1) + 1) \% 3 = 0 \implies q_0$.
- From q_2 (rem 2): on 0 $\rightarrow (2(2) + 0) \% 3 = 1 \implies q_1$; on 1 $\rightarrow (2(2) + 1) \% 3 = 2 \implies q_2$.

Step 3: State Transition Table

State (Q)	Input: 0	Input: 1	Mathematical Meaning
$\rightarrow *q_0$	q_0	q_1	Remainder 0 (Accept)
q_1	q_2	q_0	Remainder 1
q_2	q_1	q_2	Remainder 2

Final Verified Automaton $M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_0\})$

Quick Recall

For divisibility by k , construct k remainder states $\{q_0, \dots, q_{k-1}\}$. Transition formula for binary (base 2): $\delta(q_r, b) = q_{(2r+b) \pmod{k}}$.

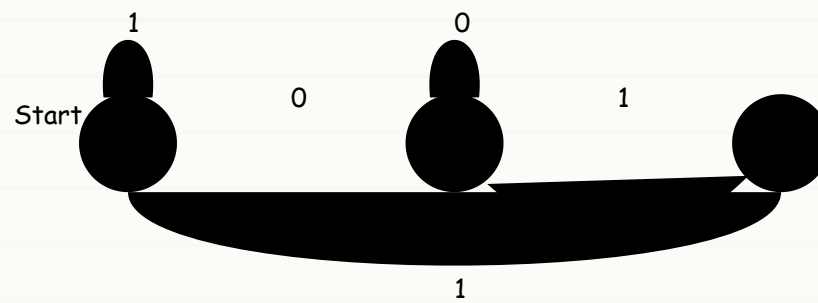


Figure 1.1: State Transition Diagram for DFA Accepting Strings Ending in '01' over $\{0, 1\}$

State q_0 is start; q_1 indicates trailing '0'; q_2 is the double-circle accept state reached upon input '1'.

 **Interactive Tool Available Online:**  [Interactive DFA State Transition Simulator](#). Try the interactive step-by-step calculator and visualizer on the live site notes page.

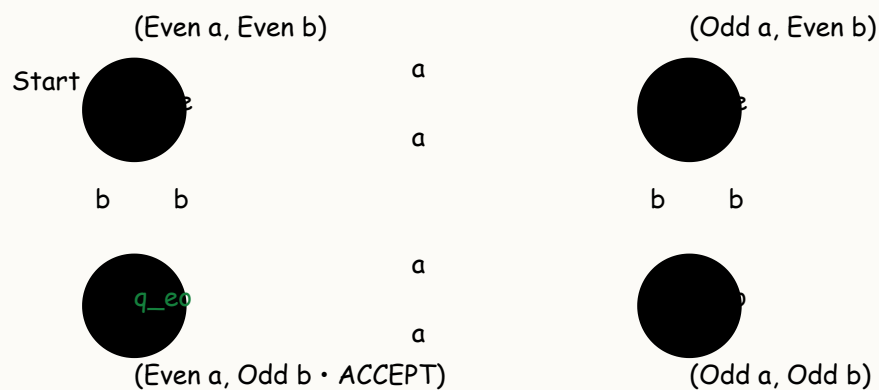


Figure 1.2: Cross-Product Parity DFA for Even number of a's and Odd number of b's

Ramaiah SEE Jan 2026 Q1(b)(iv) Solved: 4 states representing (even/odd a, even/odd b) parity cross-product. Accept state is q_{eo} .

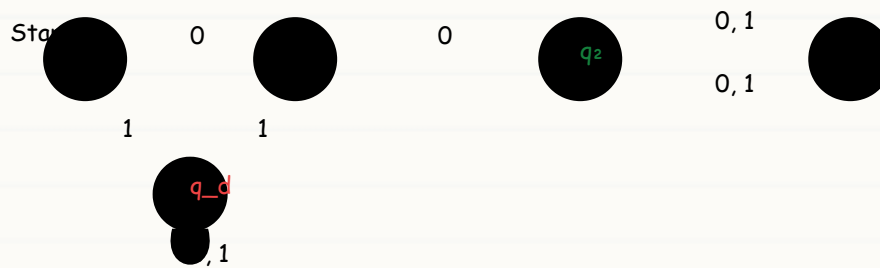


Figure 1.3: DFA Accepting Strings of Even Length Beginning with '00'
(Ramaiah CIE-1 Q3.a.i)

State q_d is a dead/trap state for strings starting with '1' or '01'. States q_2 and q_3 alternate parity for subsequent symbols.

Nondeterministic Finite Automata (NFA)

§ 4

A **Nondeterministic Finite Automaton (NFA)** relaxes the determinism constraints of DFAs. For a given state and input symbol, an NFA may transition to *zero, one, or multiple* next states simultaneously.

FORMAL 5-TUPLE DEFINITION OF AN NFA

An NFA is a 5-tuple:

$$N = (Q, \Sigma, \delta, q_0, F)$$

Where the transition function maps to the **power set** of states:

$$\delta : Q \times \Sigma \rightarrow 2^Q \quad (\text{or } \mathcal{P}(Q))$$

A string w is accepted by an NFA if *at least one* computational path leading from q_0 upon reading w halts in an accepting state $q_f \in F$.

Comparison: DFA vs. NFA

Dimension	Deterministic Finite Automata (DFA)	Nondeterministic Finite Automata (NFA)
Transition Range	$\delta(q, a) \in Q$ (exactly one state)	$\delta(q, a) \subseteq Q$ (subset of states, can be $\emptyset$)
ϵ -Transitions	Not permitted	Permitted in ϵ -NFA
Design Simplicity	Requires tracking all possible inputs from every state	Extremely compact and intuitive to design
Computational Power	Recognizes Regular Languages	Identical power (Recognizes Regular Languages)
Execution Simulation	$O(w)$ linear time in hardware/software	Requires backtracking or set simulation ($O(Q ^2 \cdot w)$)

Quick Recall

NFA transition function outputs a subset: $\delta : Q \times \Sigma \rightarrow 2^Q$. DFAs and NFAs recognize the *exact same family of languages* (Regular Languages).

ϵ -NFA & ϵ -Closure

§ 5

An ϵ -NFA allows state transitions to occur spontaneously *without consuming any input symbol* (ϵ -transitions).

DEFINITION: ϵ -CLOSURE

For a state $q \in Q$, the ϵ -closure of q (denoted $\epsilon\text{-CLOSURE}(q)$) is the set of all states reachable from q by traversing zero or more consecutive ϵ -transitions:

1. $q \in \epsilon\text{-CLOSURE}(q)$ (Every state belongs to its own ϵ -closure).
2. If $p \in \epsilon\text{-CLOSURE}(q)$ and $r \in \delta(p, \epsilon)$, then $r \in \epsilon\text{-CLOSURE}(q)$.

For a set of states $S \subseteq Q$:

$$\epsilon\text{-CLOSURE}(S) = \bigcup_{q \in S} \epsilon\text{-CLOSURE}(q)$$

Quick Recall

ϵ -transitions consume no input. $\epsilon\text{-CLOSURE}(q)$ finds all states reachable via ϵ -edges alone (always includes q itself).

Equivalence of DFA and NFA: Subset Construction Algorithm

§ 6

Rabin and Scott proved that every NFA can be converted into an equivalent DFA that accepts the exact same language. The conversion algorithm is called the **Subset Construction Algorithm** (or Powerset Construction).

THE SUBSET CONSTRUCTION ALGORITHM

Given NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, construct DFA $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$:

1. **DFA Start State:** $q_D = \{q_0\}$ (or $\epsilon\text{-CLOSURE}(q_0)$ for $\epsilon\text{-NFA}$).
2. **DFA States (Q_D):** Subsets of Q_N reachable from the start state ($Q_D \subseteq 2^{Q_N}$). Maximum theoretical states: $2^{|Q_N|}$.
3. **Transition Function:** For any DFA state $S = \{p_1, \dots, p_k\}$ and input $a \in \Sigma$:

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$$

4. **Accepting States (F_D):** Any subset $S \in Q_D$ that contains at least one accepting state of the NFA:

$$F_D = \{S \in Q_D \mid S \cap F_N \neq \emptyset\}$$

Solved Problem 3 · Core Exam Problem

Subset Construction

Convert the following NFA $N = (\{p, q, r\}, \{0, 1\}, \delta, p, \{r\})$ into an equivalent DFA:

NFA State	Input 0	Input 1
$\rightarrow p$	$\{p, q\}$	$\{p\}$
q	$\emptyset$	$\{r\}$
$*r$	$\emptyset$	$\emptyset$

Show the reachable state subsets, DFA transition table, and state diagram.

Step 1: Start State Formulation

DFA Start State: $A = [p]$.

Step 2: Trace Transitions from Reachable Subsets

• From state $[p]$:

$$\delta_D([p], 0) = \delta_N(p, 0) = [p, q] \quad (\text{New state } B)$$

$$\delta_D([p], 1) = \delta_N(p, 1) = [p] \quad (\text{State } A)$$

• From state $[p, q]$:

$$\delta_D([p, q], 0) = \delta_N(p, 0) \cup \delta_N(q, 0) = \{p, q\} \cup \emptyset = [p, q] \quad (\text{State } B)$$

$$\delta_D([p, q], 1) = \delta_N(p, 1) \cup \delta_N(q, 1) = \{p\} \cup \{r\} = [p, r] \quad (\text{New state } C)$$

• From state $[p, r]$:

$$\delta_D([p, r], 0) = \delta_N(p, 0) \cup \delta_N(r, 0) = \{p, q\} \cup \emptyset = [p, q] \quad (\text{State } B)$$

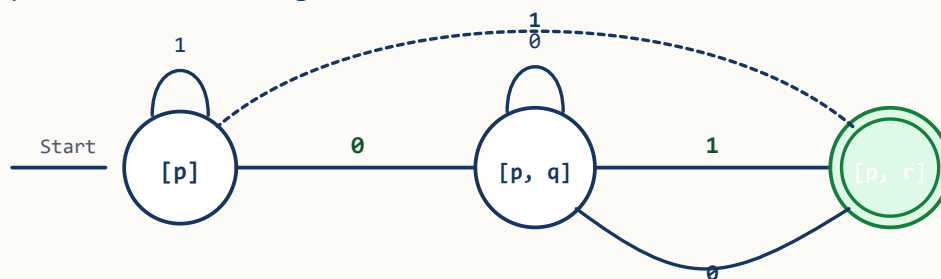
$$\delta_D([p, r], 1) = \delta_N(p, 1) \cup \delta_N(r, 1) = \{p\} \cup \emptyset = [p] \quad (\text{State } A)$$

No new states generated. Total reachable DFA states = 3 ($[p]$, $[p, q]$, $[p, r]$).

Step 3: DFA State Transition Table

DFA State Subset	Input 0	Input 1	Accepting Status
$\rightarrow [p]$	$[p, q]$	$[p]$	Non-accepting
$[p, q]$	$[p, q]$	$[p, r]$	Non-accepting
$*[p, r]$	$[p, q]$	$[p]$	Accepting ($r \in [p, r]$)

Step 4: DFA State Diagram



Final Verified Equivalent DFA QD = $\{[p], [p, q], [p, r]\} \mid q_0 = [p] \mid \text{FD} = \{[p, r]\}$ (3 states, 0 dead states)

Quick Recall

Subset construction creates DFA states representing sets of NFA states. If an NFA has n states, the DFA has at most 2^n states (in practice, far fewer reachable states).

Conversion of ϵ -NFA to DFA

§ 7

Converting an ϵ -NFA to a DFA applies the same subset construction algorithm, with transitions filtered through ϵ -closures:

ϵ -NFA TO DFA TRANSITION RULE

1. DFA Initial State:

$$q_D = \epsilon\text{-CLOSURE}(q_0)$$

2. For each generated DFA state S and input symbol $a \in \Sigma$:

$$\delta_D(S, a) = \epsilon\text{-CLOSURE} \left(\bigcup_{p \in S} \delta(p, a) \right)$$

3. Any DFA state containing at least one state from F_N is accepting.

Quick Recall

For ϵ -NFA conversion, take the ϵ -closure *before* and *after* reading each symbol:

$$\delta_D(S, a) = \epsilon\text{-CLOSURE}(\delta(S, a)).$$

Real-World Applications of Finite Automata

§ 8

Finite automata are foundational to practical computer science systems:

- **Compiler Lexical Analysis (Scanners):** Tools like `lex` and `flex` convert regular expressions describing identifiers, keywords, numbers, and operators into DFAs to tokenize source code in $O(n)$ time.
- **Text Search & Pattern Matching:** The UNIX `grep` utility and the Knuth-Morris-Pratt (KMP) / Aho-Corasick algorithms construct finite state automata to locate substrings in linear time without backtracking.
- **Network Protocol Verification:** TCP connection handshakes (SYN, SYN-ACK, ACK, ESTABLISHED, FIN_WAIT) are modeled as DFAs to prevent deadlocks and enforce protocol correctness.
- **Hardware Design & Sequential Circuits:** Digital logic controllers (traffic lights, vending machines, microprocessors) are designed as synchronous Mealy/Moore finite state machines.

Quick Recall

Applications: Lexical tokenizers (`lex/flex`), text search (`grep/KMP`), network protocols (TCP FSM), and digital circuit design.

Unit 1 Quick Revision Sheet

Key formal definitions, conversion rules, and design patterns for rapid exam recall.

DFA 5-Tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$\delta : Q \times \Sigma \rightarrow Q$$

Deterministic: exactly 1 exit transition per symbol.

NFA 5-Tuple

$$N = (Q, \Sigma, \delta, q_0, F)$$

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

Nondeterministic: 0, 1, or multiple exit transitions.

Subset Construction

Start: $\{q_0\}$ (or $\epsilon\text{-CLOSURE}(q_0)$)

$$\delta_D(S, a) = \bigcup_{p \in S} \delta(p, a)$$

Accept: any subset containing an NFA final state.

Divisibility Automata

k states for mod k .

Binary transition: $\delta(q_r, b) = q_{(2r+b) \pmod k}$.

 ϵ -Closure Rule

All states reachable via ϵ -transitions alone.

Always includes the state itself: $q \in \epsilon\text{-CLOSURE}(q)$.

State Bound

An n -state NFA converts to a DFA with at most 2^n states.

Important Exam Questions

Curated questions mapped to semester examination patterns.

2-Mark Questions **Practice Problem:**

Q1. Define an alphabet and give an example.

Ans

An alphabet is a finite, non-empty set of symbols, typically denoted by Σ .

Example: The binary alphabet $\Sigma = \{0, 1\}$.

 Practice Problem:

Q2. Define the extended transition function of a DFA.

Ans

The extended transition function $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ processes an entire string w recursively:

$$\hat{\delta}(q, \epsilon) = q, \quad \hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$$

Where $x \in \Sigma^*$ and $a \in \Sigma$.

 Practice Problem:Q3. If an NFA has n states, what is the maximum number of states in its equivalent DFA?**Ans**

The maximum number of states in the equivalent DFA is 2^n (the size of the power set $\mathcal{P}(Q)$).

5-Mark Questions **Practice Problem:**

Q4. Differentiate between DFA and NFA with transition diagrams.

Ans

Feature	DFA	NFA
Transition Function	$\delta : Q \times \Sigma \rightarrow Q$	$\delta : Q \times \Sigma \rightarrow 2^Q$
Outgoing Transitions	Exactly one per input symbol	Zero, one, or multiple transitions
ϵ -moves	Not allowed	Allowed in ϵ -NFA
String Acceptance	Unique path ends in F	At least one path ends in F

10-Mark Questions

**Practice Problem:**

Q5. State the Subset Construction algorithm and prove the equivalence of DFA and NFA.

Ans

Refer to Section 6 and Solved Problem 3 for the detailed step-by-step mathematical proof and complete worked conversion example.

Solved Questions from Past CIE-1 & SEE Examination Papers

■ ARCHIVE

The following examination questions have been transcribed from the official Ramaiah Institute of Technology question papers in notes/toc/practice/toc-cie-1-2-see.pdf (Internal Assessment - I, Oct 2025 and Semester End Examination, Jan 2026), solved with complete formal 5-tuples and intermediate steps:

Ramaiah Internal Assessment - I (Oct 2025) · Q 1(a) [7 Marks · Blooms L1, CO1]

"Define the following terms with examples: i) Alphabet, ii) String, iii) Epsilon, iv) Kleene Star, v) Language, vi) Finite Automata."

- **i) Alphabet (Σ):** A non-empty, finite set of symbols. *Example:* Binary alphabet $\Sigma = \{0, 1\}$; English lowercase alphabet $\Sigma = \{a, b, \dots, z\}$.
- **ii) String / Word (w):** A finite sequence of symbols chosen from an alphabet Σ . The length of a string, denoted $|w|$, is the number of symbol occurrences in w . *Example:* For $\Sigma = \{0, 1\}$, $w = 01101$ has length $|w| = 5$.
- **iii) Epsilon (ϵ or Λ):** The empty string, representing the unique sequence containing zero symbols. Its length is $|\epsilon| = 0$. For any string w , $\epsilon w = w\epsilon = w$.
- **iv) Kleene Star (Σ^*):** The set of all possible strings of all lengths (including the empty string ϵ) formed over alphabet Σ : $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$. *Example:* If $\Sigma = \{a\}$, then $\Sigma^* = \{\epsilon, a, aa, aaa, \dots\}$.
- **v) Language (L):** Any subset of Σ^* ($L \subseteq \Sigma^*$). A language can be finite or infinite. *Example:* $L = \{0^n 1^n \mid n \geq 1\} \subset \{0, 1\}^*$.
- **vi) Finite Automata (FA):** A mathematical model of a computing machine with a finite set of internal states, an input tape, and a state transition function that determines whether a given input string belongs to a specified regular language ($L(M)$).

Ramaiah Internal Assessment - I (Oct 2025) · Q 2(a) [8 Marks · Blooms L3, CO1]

"Consider the following ϵ -NFA with states $\{p, q, r\}$, Start: p , Accept: $\{r\}$, and transitions: $\delta(p, \epsilon)=\{r\}$, $\delta(p, a)=\{q\}$, $\delta(p, b)=\{p, r\}$; $\delta(q, a)=\{p\}$; $\delta(r, \epsilon)=\{p, q\}$, $\delta(r, a)=\{r\}$, $\delta(r, b)=\{p\}$. (i) Compute the ϵ -closure of each state. (ii) Convert the ϵ -NFA into its equivalent DFA by applying the Lazy construction method."

Part (i): Computation of ϵ -Closures:

- $\epsilon\text{ext-closure}(p)$: From p , ϵor . From r , $\epsilon\{p, q\}$. Thus: $\epsilon\text{ext-closure}(p) = \{p, q, r\}$.
- $\epsilon\text{ext-closure}(q)$: State q has no outgoing ϵ -transitions. Thus: $\epsilon\text{ext-closure}(q) = \{q\}$.
- $\epsilon\text{ext-closure}(r)$: From r , $\epsilon\{p, q\}$. From p , ϵor . Thus: $\epsilon\text{ext-closure}(r) = \{p, q, r\}$.

Part (ii): Lazy Subset Construction for Equivalent DFA:

DFA Start State $A = \epsilon\text{ext-closure}(p) = \{p, q, r\}$. Because $r \in A$, state A is an **accepting state**.

DFA State (Subset)	Input 'a' Transition & Closure	Input 'b' Transition & Closure	Status
A = {p, q, r}	$\delta(p, a) \cup \delta(q, a) \cup \delta(r, a) = \{q\} \cup \{p\} \cup \{r\} = \{p, q, r\} \circ \mathbf{A}$	$\delta(p, b) \cup \delta(r, b) = \{p, r\} \cup \{p\} = \{p, r\} \implies \epsilon\text{ext-closure}(\{p, r\}) = \{p, q, r\} = \mathbf{A}$	Start & Accept

Remarkable Optimization: Both input 'a' and input 'b' map state $\{p, q, r\}$ right back to itself! Therefore, the reachable equivalent DFA consists of **a single state** $A = \{p, q, r\}$ with self-loops on 'a' and 'b', accepting all strings in $(a + b)^*$.

Ramaiah Internal Assessment - I (Oct 2025) · Q 3(a) [7 Marks · Blooms L3, CO1]
 "Construct DFA for the following languages: i) $L = \{w \in \{0, 1\}^* \mid w \text{ is of even length and begins with } 00\}$, ii) $L = \{w \in \{a, b\}^* \mid \text{No two consecutive characters are same}\}$, iii) $L = \{w \in \{0, 1\}^* \mid w \text{ that does not end with } 00 \text{ or } 11\}$."

1. i) **Even length beginning with '00'**: Requires 5 states $\{q_0, q_1, q_2, q_3, q_d\}$.

- Start: q_0 . On '0' oq_1 , on '1' oq_d (trap).
- q_1 : On '0' oq_2 (seen 00, length 2 is even · ACCEPT), on '1' oq_d (trap).
- q_2 : On 0, $1oq_3$ (length odd ≥ 3).
- q_3 : On 0, $1oq_2$ (length even ≥ 4 · ACCEPT). (See Figure 1.3 above).

2. ii) **No two consecutive characters are same**:

- The language consists of alternating strings: $\{\epsilon, a, b, ab, ba, aba, bab, abab, baba, \dots\}$.
- States: $\{q_{start}, q_a, q_b, q_{trap}\}$, where q_{start}, q_a, q_b are all **accepting states**.
- $\delta(q_{start}, a) = q_a, \delta(q_{start}, b) = q_b$.
- $\delta(q_a, a) = q_{trap}, \delta(q_a, b) = q_b$.
- $\delta(q_b, b) = q_{trap}, \delta(q_b, a) = q_a$.

3. iii) **Does not end with '00' or '11'**:

- Design DFA tracking last two bits, and accept all states except those representing trailing '00' or '11'.
- States: q_ϵ (start, acc), q_0 (last bit 0, acc), q_1 (last bit 1, acc), q_{00} (last two 00, reject), q_{11} (last two 11, reject).

Ramaiah Semester End Examination (Jan 2026) · Q 1(b) [8 Marks · Blooms L3, CO1]

"Draw a DFA to accept the following languages: i) $L = \{w(ab+ba) \mid w \in (a,b)^*\}$, ii)

Strings of a's and b's except those having the substring 'aab', iii) Strings of 0's and 1's having three consecutive 0's, iv) Strings of a's and b's having even number of a's and odd number of b's."

- **i) Strings ending in 'ab' or 'ba':** States $\{q_0, q_a, q_b, q_{ab}, q_{ba}\}$. Start: q_0 . Accept: $\{q_{ab}, q_{ba}\}$. On reading 'a', state transitions to q_a or q_{ba} ; on reading 'b', transitions to q_b or q_{ab} .
- **ii) Except those having substring 'aab':** Construct standard DFA for pattern 'aab' with trap accept state q_{aab} . To accept the complement, invert states: make q_0, q_a, q_{aa} accepting, and make q_{aab} non-accepting (trap).
- **iii) Strings having three consecutive 0's ('000'):** 4 states $\{q_0, q_1, q_2, q_3\}$. Start: q_0 . Accept: q_3 . On '0', advance $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$. On '1', reset to q_0 (from q_0, q_1, q_2). Once in q_3 , self-loop on 0 and 1.
- **iv) Even number of a's and odd number of b's:** 4 states $Q = \{q_{ee}, q_{oe}, q_{eo}, q_{oo}\}$. Start: q_{ee} . Accept: $F = \{q_{eo}\}$. (Complete diagram provided in Figure 1.2 above).

Practice Problems with Step-by-Step Solutions

▽ PRACTICE

Test your understanding with these exam-level automata construction problems.

Click each collapsible card to reveal the complete verified solution:

PRACTICE 1 · EVEN PARITY

Design a DFA over $\Sigma = \{a, b\}$ that accepts all strings containing an even number of 'a's and an even number of 'b's. Specify the formal 5-tuple.

 Practice Problem:

Show Step-by-Step Solution

Ans

Requires 4 states representing modulo 2 cross-product parity:

- $Q = \{q_{ee}, q_{oe}, q_{eo}, q_{oo}\}$
- $\Sigma = \{a, b\}$
- $q_0 = q_{ee}$ (0 a's and 0 b's is even-even)
- $F = \{q_{ee}\}$ (only even-even is accepted)
- δ transitions:
 - $\delta(q_{ee}, a) = q_{oe}, \quad \delta(q_{ee}, b) = q_{eo}$
 - $\delta(q_{oe}, a) = q_{ee}, \quad \delta(q_{oe}, b) = q_{oo}$
 - $\delta(q_{eo}, a) = q_{oo}, \quad \delta(q_{eo}, b) = q_{ee}$
 - $\delta(q_{oo}, a) = q_{eo}, \quad \delta(q_{oo}, b) = q_{oe}$

PRACTICE 2 · SUBSTRING RECOGNITION

Construct a minimal DFA over $\{0, 1\}$ that accepts all strings containing the substring '101'.

 Practice Problem:

Show Step-by-Step Solution

Ans

States track longest matched prefix of pattern '101':

- q_0 : No prefix matched (Start) • $\delta(q_0, 0) = q_0, \delta(q_0, 1) = q_1$
- q_1 : Matched prefix '1' • $\delta(q_1, 0) = q_2, \delta(q_1, 1) = q_1$
- q_2 : Matched prefix '10' • $\delta(q_2, 0) = q_0, \delta(q_2, 1) = q_3$
- q_3 : Matched '101' (Accept state) • $\delta(q_3, 0) = q_3, \delta(q_3, 1) = q_3$ (Self-loops once matched)

PRACTICE 3 · BINARY MODULO

Design a DFA that accepts binary strings representing numbers divisible by 3 (read most significant bit first).

 Practice Problem:

Show Step-by-Step Solution

Ans

When appending bit $b \in \{0, 1\}$ to binary number N , new value is $2N + b$.

Modulo 3 arithmetic gives remainder transitions:

- States: q_0 (rem 0, Start & Accept), q_1 (rem 1), q_2 (rem 2).
- From q_0 : $(2 \text{ times } 0 + 0) \bmod 3 = 0 \rightarrow q_0$; $(2 \text{ times } 0 + 1) \bmod 3 = 1 \rightarrow q_1$.
- From q_1 : $(2 \text{ times } 1 + 0) \bmod 3 = 2 \rightarrow q_2$; $(2 \text{ times } 1 + 1) \bmod 3 = 0 \rightarrow q_0$.
- From q_2 : $(2 \text{ times } 2 + 0) \bmod 3 = 1 \rightarrow q_1$; $(2 \text{ times } 2 + 1) \bmod 3 = 2 \rightarrow q_2$.

PRACTICE 4 · NFA SUBSET CONVERSION

Given NFA $M = (\{p, q, r\}, \{0, 1\}, \delta, p, \{r\})$ where $\delta(p, 0) = \{p, q\}$, $\delta(p, 1) = \{p\}$, $\delta(q, 1) = \{r\}$. Find all reachable states in the equivalent DFA.

 Practice Problem:

Show Step-by-Step Solution

Ans

1. Start state: $\{p\}$

2. $\delta_D(\{p\}, 0) = \{p, q\}; \delta_D(\{p\}, 1) = \{p\}$

3. $\delta_D(\{p, q\}, 0) = \{p, q\}; \delta_D(\{p, q\}, 1) = \{p, r\}$

4. $\delta_D(\{p, r\}, 0) = \{p, q\}; \delta_D(\{p, r\}, 1) = \{p\}$

extReachableDFAstates : $\{p\}, \{p, q\}, \{p, r\}$ *extwithAcceptstate* $\{p, r\}$

PRACTICE 5 · COMPLEMENTATION

Given a DFA $M = (Q, \Sigma, \delta, q_0, F)$ that accepts language L , explain how to construct a DFA M' that accepts its complement L' . Does this property hold directly for NFAs?

 Practice Problem:

Show Step-by-Step Solution

Ans

For a DFA M , construct $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ by simply swapping the accepting and non-accepting states. Because every string has exactly one computation path in a DFA, string $w \in L'$ if and only if that unique path ends in $Q \setminus F$.

Crucial Distinction: This does **NOT** work directly on NFAs! Because an NFA can have multiple computation paths for a single string, simply swapping accept and non-accept states can result in both M and M' accepting the same string. An NFA must first be converted into a DFA before complementing.

PRACTICE 6 · DEAD STATE ANALYSIS

Explain what a trap (or dead) state is in a DFA, and why an NFA does not require dead states to reject strings.

 Practice Problem:

Show Step-by-Step Solution

Ans

A **trap (dead) state** is a non-accepting state from which no accepting state can ever be reached (all outgoing transitions self-loop back to itself). In a DFA, the transition function δ is total, meaning an outgoing transition must exist for every symbol in Σ ; when a prefix invalidates the language criteria (e.g. string starting with '1' for $L = \{00w\}$), the DFA must transition to a dead state.

In contrast, an NFA transition function $\delta(q, a) \subseteq Q$ allows the empty set $\emptyset$. When no transition is defined for a symbol, the machine simply crashes/halts on that branch, rejecting the string without needing an explicit trap state.